A Framework for Evaluating and Benchmarking Concept Drift Detection Methods

Vitor Cerqueira
University of Coimbra
Coimbra, Portugal
vitorc@dei.uc.pt

Heitor Murilo Gomes
Victoria University of Wellington
Wellington, New Zealand
heitor.gomes@vuw.ac.nz

Marco Heyden
Commerzbank
Frankfurt, Germany
heydenmarco48@gmail.com

Bernhard Pfahringer
University of Waikato
Waikato, New Zealand
bernhard@cs.waikato.ac.nz

Albert Bifet
University of Waikato
Waikato, New Zealand
albert.bifet@telecom-paris.fr

Abstract

Data stream mining is fundamentally challenged by concept drift, where distributional changes can degrade model performance. Despite the proliferation of drift detection methods, progress in the field is hindered by inconsistent evaluation practices: studies rely on oversimplified synthetic data generators, adopt incompatible metrics, and lack transparency in hyperparameter selection, making fair comparisons difficult. We address this gap with a novel benchmarking framework comprising three contributions: (1) a drift simulation method that injects controlled distributional changes into real-world datasets via Monte Carlo trials, enabling supervised evaluation while preserving real-world data complexity; (2) an evaluation protocol for drift detection with timing-aware criteria, including the derivation of new metrics (e.g., F1 detection score, normalized detection time) that are comparable across streams; and (3) we advocate for a leave-one-dataset-out hyperparameter optimization protocol for drift detection methods that promotes configuration robustness across heterogeneous stream dynamics. We benchmark 14 widely used drift detection methods on 7 real-world datasets across 4 drift types (class prior, label swap, feature permutation, feature filtering), each under both abrupt and gradual transitions. Our experimental results provide insights into the strengths and weaknesses of current drift detection approaches while establishing baseline performance metrics for future research in this area. All code and experiments are publicly available.

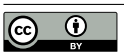
CCS Concepts

• **Computing methodologies** → **Online learning settings**; **Supervised learning**.

Keywords

Concept Drift, Data Streams, Benchmarking, Evaluation

ACM Reference Format:
Vitor Cerqueira, Heitor Murilo Gomes, Marco Heyden, Bernhard Pfahringer, and Albert Bifet. 2026. A Framework for Evaluating and Benchmarking Concept Drift Detection Methods. In *Proceedings of the 32nd ACM SIGKDD Conference on Knowledge Discovery and Data Mining V.2 (KDD '26), August 09–13, 2026, Jeju Island, Republic of Korea.* ACM, New York, NY, USA, 12 pages. https://doi.org/10.1145/3770855.3819070



This work is licensed under a Creative Commons Attribution 4.0 International License.
KDD '26, Jeju Island, Republic of Korea
© 2026 Copyright held by the owner/author(s).
ACM ISBN 979-8-4007-2259-2/2026/08
https://doi.org/10.1145/3770855.3819070

1 Introduction

Machine learning models typically rely on the assumption that the data encountered during inference follows the same distribution as the training data. However, when this distribution shifts, a phenomenon known as concept drift, model performance can degrade significantly, leading to inaccurate predictions. Therefore, reliably detecting and adapting to concept drift is crucial for maintaining the effectiveness and reliability of machine learning systems.

Detecting concept drift has long been recognized as a critical challenge in machine learning, motivating the development of numerous drift detection algorithms over the years. Notable examples include sequential analysis methods such as CUSUM and the Page-Hinkley test [26], control chart-based approaches such as DDM [15] and EDDM [1], and adaptive windowing methods such as ADWIN [5]. Yet, despite extensive research on drift detection, there is still a lack of unified evaluation practices and reproducible baselines in the field. We identify two key sources of inconsistency:

I1. **Methodology:** Existing studies are inconsistent w.r.t. evaluation metrics and protocols, hyperparameter selection strategies, datasets, and drift types.
I2. **Availability of labeled data:** Drift points—the times at which concept drifts occurred—are typically unknown in real-world data. Hence, evaluating a detector on real-world data must happen *unsupervised*. This has led to an over-reliance on synthetic data for evaluation. In synthetic data, drift points are known, and thus allow for a *supervised* evaluation. However, synthetic data and concept drifts are often unrealistic. It is thus not clear whether the obtained results hold true in the real-world [33, 36]. This, in turn, calls for an evaluation on real-world data.

These issues give rise to the research question: *How can we reliably evaluate and compare concept drift detection methods in the absence of ground truth across multiple data streams?* We tackle this question and the limitations above with the following contributions:

(1) A **drift simulation framework** that allows for controlled experiments with diverse types of drift in real-world datasets, enabling researchers and practitioners to better understand

detector behavior in different scenarios. The approach follows a Monte Carlo simulation approach where at each run predefined distribution changes are introduced into the data stream at a random point. This enables a supervised evaluation of drift detection methods while preserving the natural characteristics and inherent challenges of real-world data, addressing I2.

(2) A set of **new drift detection evaluation metrics**, including F1 detection score, drift recall, and false alarm rate, based on a **versatile definition of correct and incorrect detections**. This enables a standardized and objective comparison of drift detection methods across different datasets and drift types, addressing I1.

(3) A **practical standardized protocol for optimizing detectors** based on *leave-one-dataset-out cross-validation*: To also address I1, we propose to optimize hyperparameters of concept drift detectors on all but one dataset and use the held-out dataset for evaluation. We hypothesize that this approach encourages the optimization process to find configurations that are more robust to distinct structural dynamics. While leave-one-group-out is a standard validation approach, to our knowledge, it has not previously been applied to drift detection methods.

We apply the proposed framework to benchmark a set of state-of-the-art and widely used drift detection methods in a data stream classification scenario. We use 7 commonly used real-world data streams and introduce four different kinds of drift, namely class prior drift, class label swap drift, feature permutation drift, and feature filtering drift, each simulated with both abrupt and gradual transitions.

Our results reveal that `SEED` [23], `STEPD` [25], and `ABCD` [22] consistently outperform other detectors across distinct drift types, establishing new baseline benchmarks for the field. We further demonstrate that hyperparameter optimization using our proposed approach significantly improves detection performance over default configurations. Our implementation[1] is built on the CapyMOA Python library [19], and all code and experiment details are available for reproducibility.

2 Background

2.1 Data Streams and Concept Drift

A data stream is an infinite sequence of observations generated over time according to some unknown underlying distribution. Formally, we consider a data stream $\mathcal{S}$ as a sequence of tuples $\{(X_1, y_1), (X_2, y_2), \ldots\}$, where each X_t is an input (e.g., a feature vector) and y_t is a label.[2] At time t, the tuple (X_t, y_t) is drawn from a distribution $p_t(X, y)$, referred to as the *concept*. A concept drift has occurred if this distribution changes:

Definition 2.1 (Concept drift and drift point). We define a concept drift as a tuple (d_s, d_e) such that $p_{d_s}(X, y) \neq p_{d_e}(X, y)$ and $d_s \leq d_e$. I.e., d_s is the starting position (point of the instance where drift begins) and d_e is the ending position (point where drift completes). The width of the drift is given by $w = d_e - d_s$.

[1] https://github.com/vcerqueira/experiments-drift_evaluation
[2] W.l.o.g., we focus on categorical labels in this work.

Concept drifts can be categorized by different types of change in the distribution. Changes in $p(X)$, termed *virtual drift* (e.g. [38]), affect the input distribution but not necessarily the relationship between X and y; such drifts can be detected without labels. Changes in $p(y|X)$, usually referred to as *real drift*, alter the input-output relationship and typically require labeled data to detect. Changes may also occur in the class prior $p(y)$ or in the class-conditional feature distribution $p(X|y)$. These categories are not mutually exclusive; for instance, a shift in $p(X|y)$ will often also manifest as a change in $p(X)$.

Drifts can also be categorized by how they evolve over time [17]. In *abrupt* (or sudden) drifts, the distribution shifts instantaneously from one concept to another. In *gradual* drifts, two distinct concepts coexist during a transition period, with observations increasingly drawn from the new concept. In *incremental* drifts, a single concept evolves continuously over time without distinct before-and-after states. For sudden or abrupt drifts, $d_s = d_e$ and thus $w = 0$, while gradual and incremental drifts are characterized by $d_s < d_e$ and $w > 0$. While incremental drifts are relevant, our framework focuses on abrupt and gradual drifts.

2.2 Detection Methods

Several methods have been proposed to detect concept drift in data streams. Most of these have been designed to track the performance (e.g. error rate) of a predictive model and detect whenever it worsens significantly. Such methods typically follow an approach based on either sequential analysis, control charts, or distribution monitoring. Sequential analysis approaches work by accumulating statistics over time and comparing them to a threshold. CUSUM or Page-Hinkley [26] methods are two prominent examples following this approach. Control charts leverage the binomial distribution to define confidence intervals for the error rate, with DDM [15] or HDDM [14] being two well-known methods. Distribution monitoring approaches work by comparing the current distribution to a reference distribution. ADWIN [5], which detects significant changes in a distribution's mean, is a well-known method following this approach. We refer to the seminal survey by Gama et al. [17] for a comprehensive review on these approaches.

In domains with considerable verification delay (the time it takes to obtain a label for a given instance), monitoring performance is impractical. In effect, other approaches track indicators that can hint to a possible performance degradation and can be monitored unsupervised. Examples of such indicators are the distribution of features (e.g. [11, 22]), the distribution of predictions [27], or an auxiliary compression loss [8]. The assumption underlying these methods is that drift in the feature space or the distribution of predictions indicate a potential performance degradation.

2.3 Evaluating Drift Detection Methods

Reliably evaluating and comparing concept drift detectors requires solving three interrelated sub-problems: (1) obtaining ground truth drift points in realistic data, (2) measuring performance in a way that is comparable across datasets, and (3) selecting hyperparameters without overfitting to a single stream. We organize the related work around these challenges.

Drift point ground truth. Evaluating whether a detector correctly identifies drift points requires knowing *when* drifts occurred. Two paradigms have emerged in response. In *proxy evaluation*, detectors are assessed indirectly by measuring whether retraining a model upon detection improves predictive performance [8, 11]. While practical, this conflates detector quality with model adaptability and does not reveal detection accuracy or timing. *Supervised evaluation* instead relies on known drift points, typically obtained from synthetic data generators that switch between predefined concepts at fixed times [4, 20, 30]. Because ground truth is available by construction, one can directly measure detection accuracy, delay, and false alarm rates. However, synthetic streams often involve simplified distributions and unrealistic drift dynamics, leaving it unclear whether conclusions transfer to real-world settings [33, 36]. No prior work provides a principled way to perform supervised evaluation on *real-world* data—a gap our drift simulation framework addresses.

Comparability of evaluation metrics. Even when ground truth is available, existing metrics hinder fair cross-dataset comparison. Bifet [4] proposed three widely used metrics for supervised evaluation:

- Mean Time between False Alarms (MTFA): the average time span between consecutive false alarms before the true change point.
- Mean Detection Time (MDT): the average delay between a drift and its detection.
- Missed Detection Ratio (MDR): the proportion of drifts that go undetected.

These can be averaged over multiple runs for more robust estimates [7, 12]. However, MTFA and MDT are highly dependent on stream length and drift spacing, causing their values to fluctuate across datasets and precluding meaningful cross-dataset comparison. Other works [12, 30] have applied similar metrics under different terminology, further evidencing the lack of standardization. To mitigate dataset dependence, Heyden et al. [22] proposed an F1 score for drift detection, defining true positives, false negatives, and false positives based on whether a drift was detected before the next change occurred. While F1 improves cross-dataset comparability, its formulation ignores the temporal aspect: a detector with unacceptable delay can still achieve perfect F1. Our timing-aware metrics address this gap by normalizing for stream characteristics while penalizing detection delay.

Configuration of drift detectors. Beyond metrics, the hyperparameter configuration of drift detectors poses a reproducibility challenge. Most works report specific configurations without clear guidelines on how they were obtained, often relying on tacit expert knowledge or tuning on the same dataset used for evaluation [20]. This risks overfitting to specific stream dynamics and prevents fair comparison across studies. Our leave-one-dataset-out cross-validation protocol addresses this by optimizing hyperparameters on held-out datasets, preventing data leakage and encouraging configurations that generalize across diverse streams.

Previous benchmarks [12, 20, 30] each suffer from one or more of the above sub-problems: they rely on synthetic data for ground truth (I2), use dataset-dependent metrics (I1), or lack a principled hyperparameter protocol (I1). Our work is the first to jointly address all three challenges within a unified framework.

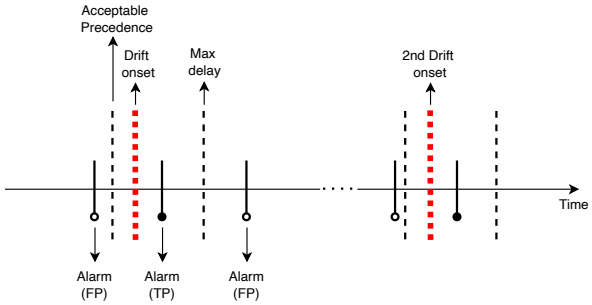


Figure 1: Drift detection evaluation criteria. The acceptable detection window is defined as the interval $[d_s - \delta_{pre}, d_e + \delta_{max}]$. Any detection falling within this window is considered a true positive, while detections outside this window are counted as false positives.

3 Drift Detection Evaluation Framework

This section introduces a systematic framework for evaluating concept drift detection methods, including definitions of detection correctness, a set of evaluation metrics derived from these definitions, and an approach for hyperparameter tuning of detectors.

3.1 Evaluation Criteria

We start by establishing clear criteria for what constitutes a correct detection, which is a key aspect for evaluating detectors in supervised settings. Recall our definition of concept in Definition 2.1: An ideal drift detector would detect data distribution changes accurately and with zero detection delay. In practice, however, a detector typically needs to first observe a certain amount of data before accurate drift detection is possible. In general, the more data a drift detector observes, the more accurate it should be. At the same time, however, this will lead to a larger detection delay. Optimizing this tradeoff is non-trivial and application-dependent.

To account for this trade-off, we introduce an evaluation criteria where a detection is only considered accurate if it occurs within a *reasonable* time window around the actual drift point. Let δ_{max} denote the maximum acceptable delay parameter, representing the maximum number of instances after d_e within which a detection must occur before the drift is considered missed. Similarly, let δ_{pre} denote the acceptable precedence parameter, representing the maximum number of instances before d_s where an early detection can be considered valid. This precedence parameter accounts for cases where detectable changes in the data stream may precede the formal drift point. For instance, some external event may cause and precede a change in the data distribution and lead to performance degradation. While the data distribution change defines the concept drift, the preceding event is a key factor for its detection.

We therefore define the acceptable detection window as an interval $[d_s - \delta_{pre}, d_e + \delta_{max}]$, illustrated in Figure 1. Any detection falling

within this window is considered a true positive, while detections outside this window are counted as false positives.

For gradual drifts ($d_s < d_e$), we anchor the acceptable delay at d_e rather than d_s. This choice ensures that the entire gradual transition falls within the acceptable window, and that δ_{max} has a consistent interpretation, i.e. maximum delay after drift completion, regardless of drift width. Note that detections occurring during the gradual transition (between d_s and d_e) are inherently valid, as they fall within the acceptable window.

In effect, the criteria are:

- **TP (True Positive)**: A concept drift occurred and was detected within the acceptable detection window $[d_s - \delta_{pre}, d_e + \delta_{max}]$.
- **FN (False Negative)**: A concept drift occurred but was not detected within the acceptable detection window $[d_s - \delta_{pre}, d_e + \delta_{max}]$).
- **FP (False Positive)**: An alarm was triggered outside any acceptable detection window $[d_s - \delta_{pre}, d_e + \delta_{max}]$

Remark (True Negative). The concept of true negative (TN) is not well defined in the context of drift detection, due to the continuity of the data stream and the fact that there are infinitely many possible moments where drift does not occur [13].

Remark (Precedence Parameter). The value of the precedence parameter δ_{pre} is situational. For example, when introducing synthetic drift points, the distribution changes can only be detected after the drift onset, so δ_{pre} should be 0 in such cases.

3.2 Evaluation Metrics

Building upon the evaluation criteria defined above, we establish several metrics to assess drift detector performance from various perspectives: accuracy, timeliness, and reliability:

- **Precision**: The proportion of detected alarms that were correct ($TP/(TP+FP)$). This metric indicates how reliable the detector's alarms are. Higher precision means most alarms correspond to actual drifts rather than false positives.
- **Recall**: The proportion of actual drifts that were correctly identified ($TP/(TP+FN)$). This measure quantifies how well the detector captures actual drift events.
- **Episode Recall**: The proportion of actual drift episodes that were correctly identified. By definition, recall counts multiple detections within the acceptable detection window as multiple TPs. Episode Recall addresses this limitation by counting multiple alarms within the same detection window as a single detection. It provides a more realistic measure of drift detection coverage.
- **F1 Score**: The harmonic mean of precision and recall.
- **Normalized Detection Time (NDT)**: The average number of instances between the start of a drift (d_s) and its detection, normalized by the maximum acceptable delay δ_{max}. This metric expresses detection delay as a proportion of the acceptable window, enabling comparisons across datasets with different δ_{max} values. NDT is computed only over detected drifts (missed drifts are excluded), so values range from 0 (immediate detection) to 1 (detection at the deadline). False negatives are not covered in this metric to avoid arbitrarily large penalties (i.e. values >> 1);
- **Alarm Rate (AR)**: The total number of alarms triggered per unit of time, calculated as $\frac{\text{Total alarms}}{\text{Total instances}} \cdot F$. By default, we use F instances as the unit of time. This measures how frequently the detector signals drifts, regardless of correctness. The factor F depends on the velocity of the data stream.
- **False Alarm Rate (FAR)**: The number of false positive alarms per unit of time, calculated as $\frac{\text{FP}}{\text{Total instances}} \cdot F$. This is particularly important as frequent false alarms can lead to unnecessary model updates and reduced system efficiency and trust.

Overall, these metrics provide a complementary view on drift detector performance. For example, NDT quantifies detection latency only for successfully detected drift episodes; it does not penalize missed detections. For this reason, NDT should always be interpreted alongside Episode Recall, which captures the proportion of drift episodes that were detected within the acceptable window.

3.3 Hyperparameter Optimization

Default hyperparameters for drift detectors are often tuned w.r.t. the specific datasets used in the original publications. However, using the same dataset for optimizing and evaluating the detector can cause overfitting and lead to overly optimistic performance estimates reported in the respective papers. To solve this issue, we propose a *leave-one-dataset-out* cross-validation approach for optimizing the hyperparameters of drift detectors.

For each dataset held out for evaluation, the remaining datasets serve as the optimization set. A search over the detector's hyperparameter space is conducted, selecting the configuration that maximizes a chosen performance metric (e.g., F1 detection score) averaged across all scenarios in the optimization set. The detector is then evaluated on the held-out dataset using the selected configuration.

Leave-one-dataset-out cross-validation ensures that detector parameters are optimized on data different from the evaluation data, leading to more reliable performance estimates while at the same time getting the most out of detectors performance. The approach can be coupled with any search strategy, such as random search [3].

4 Drift Simulation Framework

We now introduce our method for injecting systematically controllable concept drift into real-world data.

4.1 Simulating Drifts

Our approach adopts a semi-synthetic methodology: given a real-world data stream, our approach injects synthetic concept drift at specified points.

4.1.1 Simulation Process. Our drift simulation framework follows the prequential learning paradigm [16] and consists of four steps:

(1) *Random shuffling:* Real-world data streams, denoted $\mathcal{S}$, may already contain unknown concept drifts, which could confound the results of controlled experiments. To minimize this risk, we randomly shuffle the observations in $\mathcal{S}$, producing a permuted stream $\mathcal{S}_\pi$. This step removes pre-existing temporal dependencies (e.g., seasonality) by design, bringing

it closer to i.i.d., and laying the ground for controlled drift injection with known change points.

(2) *Randomized drift onset:* In each prequential evaluation run, we select a single drift onset point d_s at random, subject to two buffer regions. The first buffer, at the beginning of $\mathcal{S}_\pi$, ensures the classifier has sufficient training data and enough pre-drift samples for evaluating false alarms. The second buffer, at the end of the stream, provides ample post-drift samples for assessing detection performance. Once d_s is established, all subsequent instances are transformed using a designated drift function (as detailed in Section 4.2).

(3) *Monte Carlo trials:* The above process (shuffling, selecting a randomized drift point, and applying drift) is repeated k times. This repetition allows for robust statistical assessment of detector performance across multiple runs of the prequential workflow.

(4) *Abrupt vs. gradual drift:* For abrupt drifts ($w = 0$), the drift transformation is applied to all instances from the drift onset onwards. For gradual drifts ($w > 0$), the transformation is applied probabilistically: the probability that an instance at time t is drawn from the transformed distribution increases linearly from 0 at d_s to 1 at d_e, computed as $p(t) = \frac{t-d_s}{w}$. After d_e, all instances are transformed.

4.1.2 Instance transformation and evaluation. Beyond the workflow described above, the core mechanism for simulating drift is the transformation and evaluation of data instances after the drift onset. Figure 2 illustrates this process. Let M denote a predictive model (here, a classifier), g the drift simulation function that applies the selected drift pattern, and $I = (X, y)$ an instance from $\mathcal{S}_\pi$.

For each incoming instance $I = (X, y)$, the framework determines whether drift should be applied based on the current position relative to the drift onset d_s. Before d_s, instances pass through unchanged. After d_s, the drift simulation function transforms I into $I' = g(I)$ according to the specified drift pattern. The model M then generates a prediction $\hat{y}$ for the (possibly transformed) instance. Most of the drift detectors then monitor M's prediction error over time. However, other inputs are possible, such as the feature vectors [22], or an auxiliary compression error [8]. After inference, the model M is trained on the instance I, or the transformed instance I'. The complete drift simulation framework is illustrated in Algorithm 1 in appendix.

4.2 Drift Types

The previously introduced simulation framework uses a generalized drift function g. This allows modeling different types of distribution changes. We now provide concrete examples of g, illustrating how to generate the types mentioned in Section 2.1. See Appendix A.2 for the respective pseudocode.

4.2.1 Class Prior Drift (Algorithm 2). Class prior drift simulates a change in the class distribution by altering the frequency of a specific target class. After the drift onset, instances belonging to a pre-selected label y_{sel} are randomly dropped from the stream with probability p_{skip}, effectively reducing the occurrence of that class. When an instance is dropped, the prequential workflow skips it entirely (i.e. no training or inference is performed on it), proceeding to the next observation. Zliobaite [38] evaluated this drift type in the context of change detection with delayed labeling, though in a non-randomized setting (e.g. fixing the proportions of the classes on specific datasets, predefined drift onset), while we create a more general framework with randomized parameters.

4.2.2 Class Label Swap Drift (Algorithm 3). Class label swap drift simulates a change in the classification function by relabeling instances of a specific class. After the drift onset, any instance originally belonging to a selected label y_{sel} has its true label changed to a different target label y_{swap}, while its feature vector remains unchanged. This represents a scenario where the underlying classification rule changes, with the same input now corresponding to a different output.

4.2.3 Feature Permutation Drift (Algorithm 4). Feature permutation drift introduces a structural change in the feature space by reordering the input features. After the drift onset, the features within each instance's vector x are rearranged according to a fixed, randomly generated permutation σ. The class label remains unchanged. This simulates scenarios where the relative importance or ordering of features shifts. Zliobaite [38] also evaluated this drift in a class-conditional setting (permutation of features within a specific class), with the class and the permutation of features being hand-picked for each dataset.

4.2.4 Feature Filtering Drift (Algorithm 5). Feature filtering drift simulates a change in the marginal distribution of a feature by conditionally filtering instances. After the drift onset, instances are checked against a threshold condition: if the value of a randomly selected numeric feature exceeds a predefined threshold τ, the instance is dropped from the stream; otherwise, it passes through unchanged. When an instance is dropped, the prequential workflow skips it entirely and proceeds to the next observation. This simulates scenarios where, due to policy changes, sensor limitations, or data collection constraints, certain feature values are no longer observed. In this work, we consider τ to be the median of the feature values observed before the drift onset.

4.2.5 Discussion. The four drift types described above represent common categories of distribution change that can occur in real-world data streams. They are not intended to be exhaustive; many other forms of drift exist, such as noise injection, heteroskedasticity, or temporal dependencies [37]. Additionally, incremental drifts, where the distribution shifts continuously over an extended period [34], are also relevant but not considered in this work. Our selection focuses on distinct, interpretable drift patterns that allow for controlled experimentation.

Each drift type involves random parameter choices (e.g., which class to affect, which feature to filter). This variability means that a single run may not fully characterize detector performance. The Monte Carlo approach described earlier addresses this by repeating the evaluation across multiple runs with different random configurations, leading to more robust performance estimates. For simplicity, each Monte Carlo run contains a single drift, whereas real-world streams may exhibit multiple ones. These scenarios can be framed as different episodes of drift events as illustrated in Figure 1.

Finally, because each drift type transforms instances from I to I' (or filters them), all four can be applied in either abrupt or gradual

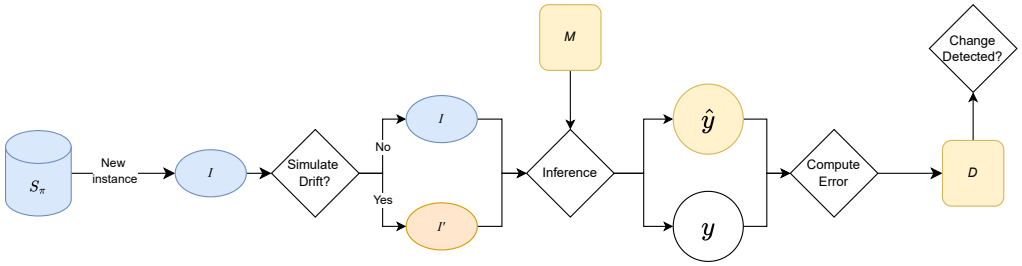


Figure 2: Inference stage of the prequential workflow with simulated drifts (using error tracking as example).

mode as described in Section 4. This flexibility allows the framework to simulate a wide range of drift dynamics.

5 Experiments

5.1 Experimental Setup

Datasets. We evaluate our approach on seven widely used real-world datasets from the USP Data Stream Repository [32]: Asfault [31], Electricity [21], Covertype [6], GasSensorArray [35], NOAA [9], Posture [11], and Rialto [24]. Detailed descriptions of each dataset are available in the repository[3].

To create controlled evaluation settings, we systematically generate semi-synthetic data streams by injecting known concept drift points into these datasets using the proposed drift simulation framework. For each dataset and drift type, we perform 50 Monte Carlo trials. In each trial, the drift onset time is randomly sampled between 50% and 80% of the total dataset length. Drift width and maximum delay parameters are dataset-specific. These were selected based on dataset length and are summarized in Table 3. The precedence parameter is fixed at zero ($\delta_{pre} = 0$) for all datasets since synthetic drifts cannot be not anticipated before their onset.

We benchmark a total of 14 change detection methods: `ADWIN` [5], `CUSUM` [26], `DDM` [15], `EDDM` [1], `EWMA` [29], `GMA` [28], `HDDMA` [14], `HDDMW` [14], `PH` [26], `RDDM` [2], `SEED` [23], `STEPD` [25], `ABCD` and `ABCD(X)` [22], and `STUDD` [8]. Most of these are based on tracking the performance of a classifier, namely the instance-wise error rate, except for `ABCD(X)` [22], which monitors the feature space and `STUDD`, which monitors an auxiliary compression loss [8]. Most of these methods follow a distribution monitoring, control chart, or sequential analysis approach. The exception is `STUDD`, which, being a meta-detector, can be coupled with any of the other methods. In this work, `STUDD` is coupled with `ADWIN`. We also note that `ABCD` can be applied both to the error rate and the feature space. We denote the latter as `ABCD(X)`. We note that our main objective is to establish a standardized evaluation protocol rather than to identify the best performing detector. Thus, we focus on well-established detection methods whose performance characteristics are already well understood.

We select the Hoeffding Tree [10] as the classifier in the experiments, a well-established algorithm in the streaming machine learning literature. While ensembles of Hoeffding Trees (e.g. [18]) have been shown to be more accurate, we focus on the Hoeffding Tree due to its computational efficiency. We use default hyperparameters on Hoeffding Tree classifier across all experiments[4].

Hyperparameter Optimization. Regarding hyperparameter optimization, we conduct the leave-one-dataset-out approach described in Section 3.3. When evaluating the detectors on each dataset, we use the remaining datasets for optimizing its hyperparameters based on the F1 detection score. The optimization is conducted using 30 iterations of random search. Each configuration is also evaluted using the Monte Carlo procedure described in Section 3.3. The configuration space for each detector is presented in Table 4 in appendix.

5.2 Results

Most of the results presented below are based on F1 score. The scores on the remaining metrics (including execution time), other classifiers and synthetic data streams, are available in the Appendix B.2. In all tables, bold (underlined) scores denote the best (second-best) detector in the respective scenario.

5.2.1 Overall scores. Tables 1 and 2 show the average rank based on F1 detection score for each drift detection method across different drift scenarios, for abrupt and gradual drifts respectively. The average rank of a detector denotes its relative position according to performance; lower values indicate better performance.

The results reveal distinct tiers of detector performance. `SEED` and `STEPD` exhibit consistently competitive F1 scores across all drift types and abruptness conditions, ranking among the top three performers in most scenarios.

Several detectors perform well under specific conditions but struggle in others. Among performance-tracking methods, `ADWIN` excels at feature filtering and class prior drifts but underperforms on feature permutation. `ABCD` achieves top-tier ranks on feature permutation and class swaps while showing weaker performance on feature filtering. `DDM` shows moderate results overall, performing best on class prior under gradual conditions. `GMA` demonstrates particular strength on feature filtering in gradual settings, achieving the best average rank for that scenario. `HDDMW` shows stable middle-tier performance (average ranks around 5 to 7) across most conditions.

The unsupervised detectors, `ABCD(X)` and `STUDD`, exhibit a characteristic pattern: reasonable or strong performance on feature-space drifts but poor results on label-based changes. `ABCD(X)` achieves

[3]https://sites.google.com/view/uspdsrepository

[4]https://capymoa.org/api/modules/capymoa.classifier.HoeffdingTree.html

Table 1: Average rank of drift detectors across different datasets for abrupt drifts

Detector	Feature Filtering	Feat. Permutation	Class Prior	Class Swap
ABCD	8.1	3.9	8.0	3.6
ABCD(X)	12.7	**1.0**	13.1	13.1
ADWIN	4.1	7.6	4.1	7.1
CUSUM	5.6	11.9	6.1	9.2
DDM	7.3	11.4	6.0	9.9
EWMA	9.7	12.9	9.4	10.9
GMA	9.0	7.7	8.7	7.3
HDDMA	7.4	5.6	6.3	5.4
HDDMW	7.1	6.9	5.8	6.0
PH	13.1	12.9	11.6	10.1
RDDM	6.6	10.2	7.9	9.1
SEED	4.4	2.7	**3.9**	1.1
STEPD	**3.4**	3.7	4.3	2.4
STUDD	6.4	6.8	9.9	9.6

near-perfect relative detection of feature permutation drifts (rank 1.0 for abrupt, 3.3 for gradual) but fails on class prior and class swap drifts. STUDD ranks second on feature filtering for gradual drifts but is less effective for label-based scenarios. This behavior is expected, as these methods cannot observe changes that only affect the target variable. Finally, PH and EWMA consistently rank among the worst performers across nearly all scenarios, with average ranks typically exceeding 10.

Table 2: Average rank of drift detectors across different datasets for gradual drifts

Detector	Feature Filtering	Feat. Permutation	Class Prior	Class Swap
ABCD	11.7	4.1	9.8	5.1
ABCD(X)	13.0	3.3	13.0	13.1
ADWIN	6.0	11.2	6.8	7.9
CUSUM	6.3	11.9	7.3	9.7
DDM	6.1	9.4	**4.7**	8.1
EWMA	11.7	12.4	10.6	12.0
GMA	**3.9**	6.6	6.1	4.7
HDDMA	9.7	7.9	7.8	10.5
HDDMW	5.5	6.3	5.2	5.4
PH	10.9	12.4	9.7	11.1
RDDM	4.7	6.4	6.6	5.3
SEED	5.9	**2.7**	5.0	1.9
STEPD	5.4	2.8	5.2	2.0
STUDD	4.2	7.6	7.1	8.1

5.2.2 Abrupt vs Gradual. Figure 3 compares abrupt versus gradual scenarios by showing the F1 score distribution for each detector. Each boxplot aggregates $4 \times 7 = 28$ data points (4 drift types $\times$ 7 datasets).

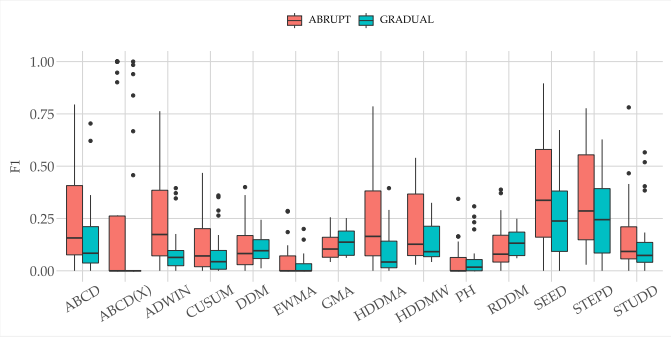


Figure 3: Distribution of F1 detection scores across dataset/drift-type pairs for each detector, comparing abrupt and gradual drift conditions.

As expected, abrupt drifts are generally easier to detect: most detectors exhibit higher median F1 scores and greater upward variance for abrupt conditions. SEED and STEPD maintain the highest medians in both conditions, though with notable performance degradation under gradual drifts.

5.2.3 F1 vs FAR. Figure 4 visualizes the trade-off between detection accuracy (F1 average rank) and reliability (FAR average rank), with detection timing (MDT average rank) encoded as dot size. Lower values indicate better performance on all three axes; an ideal detector would appear in the bottom-left corner with a small dot.

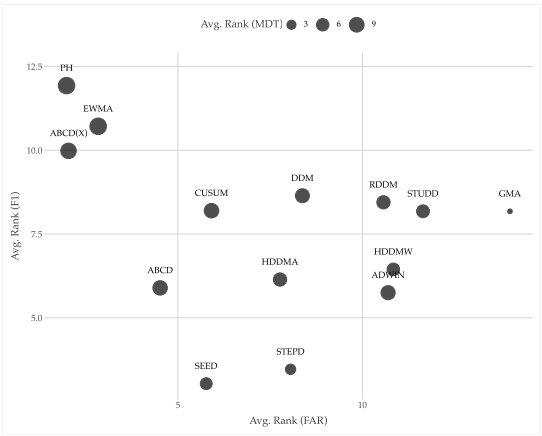


Figure 4: Trade-off between F1 average rank (y-axis, lower is better) and FAR average rank (x-axis, lower is better). Dot size encodes MDT average rank (smaller is faster detection).

SEED achieves the best F1 rank while maintaining a moderate FAR, making it the most balanced choice overall. STEPD follows closely with similar characteristics. ABCD offers a compelling alternative when minimizing false alarms is a priority; it achieves the best FAR rank among detectors with competitive F1 scores, though with slightly lower F1 performance than SEED and STEPD.

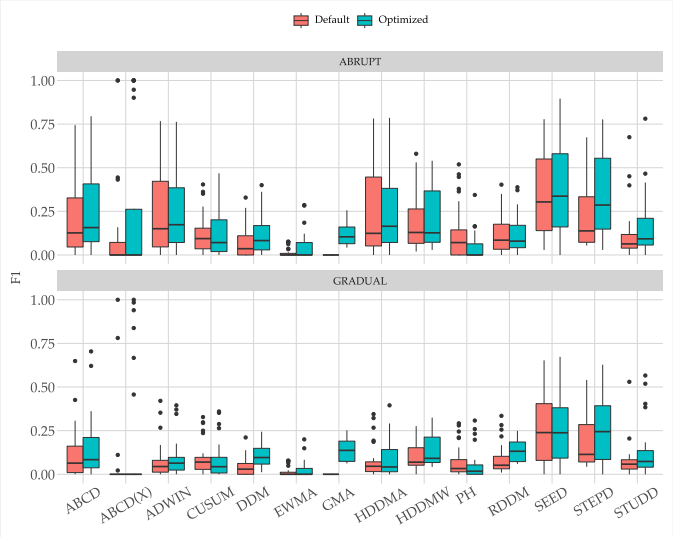


Figure 5: Distribution of F1 scores across different dataset/drift type pairs for each detection method controlling for hyperparameter optimization approach and drift abruptness.

5.2.4 Detector hyperparameter optimization. Finally, we analyse the impact of the hyperparameter optimization described in Section 3.3. The results reported so far show the scores of each detector after applying hyperparameter optimization. Figure 5 shows the distribution of F1 scores across different dataset/drift type pairs for each detection method controlling for hyperparameter optimization approach: optimized and default parameters. The results show that most approaches improve their median F1 score when optimized, apart from SEED.

5.3 Discussion

Main findings. The experiments reveal distinct patterns in detector performance. We highlight the following main findings:

(1) SEED and STEPD exhibit the most consistent performance across all drift types and abruptness conditions, ranking among the top three in most scenarios. When minimizing false alarms is a priority, ABCD offers a compelling alternative with the best FAR rank among accurate detectors, though with slightly lower F1 than SEED and STEPD.
(2) Gradual drifts are systematically harder to detect than abrupt ones. Most detectors show lower median F1 scores and reduced variance under gradual conditions, with the performance gap particularly pronounced for methods like ABCD and ABCD(X).
(3) Unsupervised detectors (ABCD(X) and STUDD) perform better on feature-space drifts (feature permutation, feature filtering) than on label-based changes (class prior, class swaps), as expected since they cannot observe target variable shifts.
(4) Hyperparameter optimization yields improvements for most detectors, with gains more pronounced for abrupt drifts. Notably, STEPD's strong performance is largely attributable to effective tuning. PH and EWMA remain ineffective regardless of configuration, suggesting fundamental limitations rather than suboptimal defaults.

These findings offer practical guidance for practitioners: SEED and STEPD are robust defaults when drift characteristics are unknown, while specialized detectors may be preferred when the drift type or false alarm tolerance is constrained.

Limitations and future work. While the experiments provide valuable insights, there are some limitations and potential future work. First, the experiments are limited to one classifier (Hoeffding tree) and four drift types simulated in 7 real-world datasets. Nonetheless, the proposed drift simulation framework is generic and can be extended with additional drift definitions, or mixtures of them. Another limitation is that the prequential workflow is simplified in the sense that it assumes immediate feedback, with labels being readily available at each step after inference. In practice, this is often not the case [8]. Finally, in our drift simulation procedure, we randomly shuffle the original data stream to eliminate any pre-existing drifts that might confound the evaluation. This approach preserves the real-world feature-level complexity of the data; however, it also removes any inherent temporal structure. Preserving temporal dependencies in future extensions of the framework would enable a more comprehensive and realistic assessment of detector performance.

6 Conclusion

This work contributes with a unified framework for evaluating and comparing change detection methods in data streams. The novelty of the framework settles on three main contributions: (1) a flexible drift simulation framework that allows for controlled experiments with various types of drift, enabling practitioners to better understand detector behavior under different drift conditions; (2) a systematic evaluation criteria to compare drift detection methods, including new evaluation metrics based on detection precision and recall, with the goal of building a common ground for the evaluation of drift detection methods; (3) an approach for hyperparameter optimization of drift detection methods, based on a leave-one-dataset-out scheme.

We applied the proposed framework to benchmark state-of-the-art and widely used drift detection methods on synthetic and real-world datasets. The experiments reveal interesting patterns in the performance of the detectors. While there is a noticeable variability in relative performance across different conditions, we highlight that SEED, STEPD and ABCD show the best overall performance across all datasets and drift types.

Acknowledgements

This work is funded by national funds through FCT – Foundation for Science and Technology, I.P., within the scope of the research unit UID/00326 - Centre for Informatics and Systems of the University of Coimbra, https://doi.org/10.54499/UID/00326/2025. Heitor Murilo Gomes acknowledges the financial support of the Marsden Fund under award number VUW2213.

References

[1] Manuel Baena-Garcıa, José del Campo-Ávila, Raul Fidalgo, Albert Bifet, Ricard Gavalda, and Rafael Morales-Bueno. 2006. Early drift detection method. In *Fourth international workshop on knowledge discovery from data streams*, Vol. 6. 77–86.

[2] Roberto SM Barros, Danilo RL Cabral, Paulo M Gonçalves Jr, and Silas GTC Santos. 2017. RDDM: Reactive drift detection method. *Expert Systems with Applications* 90 (2017), 344–355.

[3] James Bergstra and Yoshua Bengio. 2012. Random search for hyper-parameter optimization. *The journal of machine learning research* 13, 1 (2012), 281–305.

[4] Albert Bifet. 2017. Classifier concept drift detection and the illusion of progress. In *International conference on artificial intelligence and soft computing*. Springer, 715–725.

[5] Albert Bifet and Ricard Gavalda. 2007. Learning from time-changing data with adaptive windowing. In *Proceedings of the 2007 SIAM international conference on data mining*. SIAM, 443–448.

[6] Jock A Blackard and Denis J Dean. 1999. Comparative accuracies of artificial neural networks and discriminant analysis in predicting forest cover types from cartographic variables. *Computers and electronics in agriculture* 24, 3 (1999), 131–151.

[7] Vitor Cerqueira, Heitor Murilo Gomes, and Albert Bifet. 2020. Unsupervised concept drift detection using a student–teacher approach. In *International conference on discovery science*. Springer, 190–204.

[8] Vitor Cerqueira, Heitor Murilo Gomes, Albert Bifet, and Luis Torgo. 2023. STUDD: A student–teacher method for unsupervised concept drift detection. *Machine Learning* 112, 11 (2023), 4351–4378.

[9] Gregory Ditzler and Robi Polikar. 2012. Incremental learning of concept drift from streaming imbalanced data. *IEEE transactions on knowledge and data engineering* 25, 10 (2012), 2283–2301.

[10] Pedro Domingos and Geoff Hulten. 2000. Mining high-speed data streams. In *Proceedings of the sixth ACM SIGKDD international conference on Knowledge discovery and data mining*. 71–80.

[11] Denis Moreira Dos Reis, Peter Flach, Stan Matwin, and Gustavo Batista. 2016. Fast unsupervised online drift detection using incremental kolmogorov-smirnov test. In *Proceedings of the 22nd ACM SIGKDD international conference on knowledge discovery and data mining*. 1545–1554.

[12] William J Faithfull, Juan J Rodríguez, and Ludmila I Kuncheva. 2019. Combining univariate approaches for ensemble change detection in multivariate data. *Information Fusion* 45 (2019), 202–214.

[13] Tom Fawcett and Foster Provost. 1999. Activity monitoring: Noticing interesting changes in behavior. In *Proceedings of the fifth ACM SIGKDD international conference on Knowledge discovery and data mining*. 53–62.

[14] Isvani Frias-Blanco, José del Campo-Ávila, Gonzalo Ramos-Jimenez, Rafael Morales-Bueno, Agustin Ortiz-Diaz, and Yailé Caballero-Mota. 2014. Online and non-parametric drift detection methods based on Hoeffding's bounds. *IEEE Transactions on Knowledge and Data Engineering* 27, 3 (2014), 810–823.

[15] Joao Gama, Pedro Medas, Gladys Castillo, and Pedro Rodrigues. 2004. Learning with drift detection. In *Brazilian symposium on artificial intelligence*. Springer, 286–295.

[16] Joao Gama, Raquel Sebastiao, and Pedro Pereira Rodrigues. 2013. On evaluating stream learning algorithms. *Machine learning* 90, 3 (2013), 317–346.

[17] João Gama, Indrė Žliobaitė, Albert Bifet, Mykola Pechenizkiy, and Abdelhamid Bouchachia. 2014. A survey on concept drift adaptation. *ACM computing surveys (CSUR)* 46, 4 (2014), 1–37.

[18] Heitor M Gomes, Albert Bifet, Jesse Read, Jean Paul Barddal, Fabrício Enembreck, Bernhard Pfharinger, Geoff Holmes, and Talel Abdessalem. 2017. Adaptive random forests for evolving data stream classification. *Machine Learning* 106, 9 (2017), 1469–1495.

[19] Heitor Murilo Gomes, Anton Lee, Nuwan Gunasekara, Yibin Sun, Guilherme Weigert Cassales, Justin Liu, Marco Heyden, Vitor Cerqueira, Maroua Bahri, Yun Sing Koh, et al. 2025. CapyMOA: efficient machine learning for data streams in python. *arXiv preprint arXiv:2502.07432* (2025).

[20] Paulo M Gonçalves Jr, Silas GT de Carvalho Santos, Roberto SM Barros, and Davi CL Vieira. 2014. A comparative study on concept drift detectors. *Expert Systems with Applications* 41, 18 (2014), 8144–8156.

[21] Michael Harries, New South Wales, et al. 1999. Splice-2 comparative evaluation: Electricity pricing. (1999).

[22] Marco Heyden, Edouard Fouché, Vadim Arzamasov, Tanja Fenn, Florian Kalinke, and Klemens Böhm. 2024. Adaptive Bernstein change detector for high-dimensional data streams. *Data Mining and Knowledge Discovery* 38, 3 (2024), 1334–1363.

[23] David Tse Jung Huang, Yun Sing Koh, Gillian Dobbie, and Russel Pears. 2014. Detecting volatility shift in data streams. In *2014 IEEE International Conference on Data Mining*. IEEE, 863–868.

[24] Viktor Losing, Barbara Hammer, and Heiko Wersing. 2016. KNN classifier with self adjusting memory for heterogeneous concept drift. In *2016 IEEE 16th international conference on data mining (ICDM)*. IEEE, 291–300.

Table 3: Dataset summary and drift parameters

	# Samples	# Features	# Classes	Max Delay	Drift Width
Asfault	8563	62	5	500	500
Covtype	100000	54	7	2500	2500
Electricity	45312	8	2	2500	2500
GasSensor	13910	128	6	1500	1500
NOAA	18159	8	2	1500	1500
Posture	100000	3	8	2500	2500
Rialto	82250	27	10	2500	2500

[25] Kyosuke Nishida and Koichiro Yamauchi. 2007. Detecting concept drift using statistical testing. In *International conference on discovery science*. Springer, 264–269.

[26] Ewan S Page. 1954. Continuous inspection schemes. *Biometrika* 41, 1/2 (1954), 100–115.

[27] Fábio Pinto, Marco OP Sampaio, and Pedro Bizarro. 2019. Automatic model monitoring for data streams. *arXiv preprint arXiv:1908.04240* (2019).

[28] Stuart W Roberts. 2000. Control chart tests based on geometric moving averages. *Technometrics* 42, 1 (2000), 97–101.

[29] Gordon J Ross, Niall M Adams, Dimitris K Tasoulis, and David J Hand. 2012. Exponentially weighted moving average charts for detecting concept drift. *Pattern recognition letters* 33, 2 (2012), 191–198.

[30] Raquel Sebastiao and Joao Gama. 2009. A study on change detection methods. In *Progress in artificial intelligence, 14th Portuguese conference on artificial intelligence, EPIA*, Vol. 2009.

[31] Vinicius MA Souza. 2018. Asphalt pavement classification using smartphone accelerometer and complexity invariant distance. *Engineering Applications of Artificial Intelligence* 74 (2018), 198–211.

[32] V. M. A. Souza, D. M. Reis, A. G. Maletzke, and G. E. A. P. A. Batista. 2020. Challenges in Benchmarking Stream Learning Algorithms with Real-world Data. *Data Mining and Knowledge Discovery* 34 (2020), 1805–1858. doi:10.1007/s10618-020-00698-5

[33] W Nick Street and YongSeog Kim. 2001. A streaming ensemble algorithm (SEA) for large-scale classification. In *Proceedings of the seventh ACM SIGKDD international conference on Knowledge discovery and data mining*. 377–382.

[34] Yibin Sun, Heitor Murilo Gomes, Bernhard Pfahringer, and Albert Bifet. 2025. Evaluation for Regression Analyses on Evolving Data Streams. *arXiv preprint arXiv:2502.07213* (2025).

[35] Alexander Vergara, Shankar Vembu, Tuba Ayhan, Margaret A Ryan, Margie L Homer, and Ramón Huerta. 2012. Chemical gas sensor drift compensation using classifier ensembles. *Sensors and Actuators B: Chemical* 166 (2012), 320–329.

[36] Gerhard Widmer and Miroslav Kubat. 1996. Learning in the presence of concept drift and hidden contexts. *Machine learning* 23, 1 (1996), 69–101.

[37] Giacomo Ziffer, Federico Giannini, and Emanuele Della Valle. 2024. Tenet: Benchmarking Data Stream Classifiers in Presence of Temporal Dependence. In *2024 IEEE International Conference on Big Data (BigData)*. IEEE, 1187–1196.

[38] Indre Žliobaite. 2010. Change with delayed labeling: When is it detectable?. In *2010 IEEE international conference on data mining workshops*. IEEE, 843–850.

A Drift Simulation

A.1 Simulation Framework

Algorithm 1 describes the procedure for simulating drift on real-world data streams based on Monte Carlo trials.

A.2 Drift Types

Algorithms 2, 3, 4, and 5 describe the specific drift types applied in our work using the proposed simulation framework.

B Experiments

B.1 Experimental Setup

Tables 3 and 4 summarise the key parameters and configuration pool considered for the experiments.

Algorithm 1 Drift Simulation Framework

Require: Data stream $\mathcal{S}$, Drift function g, Predictive model M, Change detector D
Require: Number of trials k, Drift width w, Buffer sizes b_{start}, b_{end}
Ensure: Detection results for each trial

for $j = 1$ to k **do** ▷ Monte Carlo trials
  $\mathcal{S}_\pi \leftarrow \textsc{Shuffle}(\mathcal{S})$ ▷ Random permutation
  $d_s \leftarrow \textsc{RandomInt}(b_{start}, |\mathcal{S}_\pi| - b_{end})$ ▷ Random drift onset
  $d_e \leftarrow d_s + w$ ▷ Drift end position
  $\textsc{Reset}(M, D)$ ▷ Initialize model and detector
  for $i = 1$ to $|\mathcal{S}_\pi|$ **do**
    $I \leftarrow \mathcal{S}_\pi[i]$ ▷ Get instance (X, y)
    if $i < d_s$ **then** ▷ Before drift onset
      $I' \leftarrow I$
    else if $i \geq d_e$ **then** ▷ After drift completes
      $I' \leftarrow g(I)$
    else ▷ Gradual drift region
      $p \leftarrow (i - d_s)/w$
      $I' \leftarrow g(I)$ with probability p, else $I' \leftarrow I$
    end if
    $\hat{y} \leftarrow M.\textsc{Predict}(I'.X)$ ▷ Inference
    $e \leftarrow \textsc{Error}(\hat{y}, I'.y)$ ▷ Compute error
    $D.\textsc{Update}(e)$ ▷ Feed detector
    if $D.\textsc{Alarm}()$ **then**
      Record detection at position i
    end if
    $M.\textsc{Train}(I')$ ▷ Update model
  end for
  Store results for trial j
end for

Algorithm 2 Class Prior Drift Simulation

Require: Instance $I = (x, y)$, Selected label y_{sel}, Skip probability p_{skip}
Ensure: Instance I or Null (if skipped)

if $I.y = y_{sel}$ **then**
  Generate random value $r \sim \text{Uniform}(0, 1)$
  if $r < p_{skip}$ **then**
    return Null ▷ Instance is dropped
  end if
end if
return I ▷ Instance is kept unchanged

Algorithm 3 Class Label Swap Drift Simulation

Require: Instance $I = (x, y)$, Selected label y_{sel}, Swap label y_{swap}
Ensure: Transformed instance I'

if $I.y = y_{sel}$ **then**
  $I' = (I.x, y_{swap})$ ▷ Create instance with swapped label
  return I'
end if
return I ▷ Instance remains unchanged

Algorithm 4 Feature Permutation Drift Simulation

Require: Instance $I = (x, y)$, Permutation index vector σ
Ensure: Transformed instance I'

$x' \leftarrow x[\sigma]$ ▷ Reorder features according to σ
$I' = (x', I.y)$ ▷ Create instance with permuted features
return I'

Algorithm 5 Feature Distribution Drift Simulation

Require: Instance $I = (x, y)$, Feature index j, Threshold τ
Ensure: Instance I or Null (if filtered)

if $I.x[j] > \tau$ **then**
  return Null ▷ Instance is dropped
end if
return I ▷ Instance is kept unchanged

B.2 Results

This section presents the complete experimental results obtained from evaluating the drift detection methods across all datasets and drift scenarios. Similarly to the main content, in all tables, **bold** values indicate the best performing method for each drift type, while underlined values denote the second-best performance. Tables 5 and **??** report the average F1 detection scores for abrupt and gradual drift scenarios, respectively. Tables **??** and 8 present the average precision scores for abrupt and gradual drift scenarios, respectively. Tables 9 and 10 report the average recall scores for abrupt and gradual drift scenarios, respectively. Table 11 shows the F1 score on synthetic data streams, and Table 12 reports the F1 score across different classifiers. Below, we provide a detailed analysis of each set of results.

Decomposing the F1 scores into precision and recall (Tables **??**–10) reveals different profiles across detectors and clarifies how some methods achieve similar F1 scores through different behaviors. For example, GMA exhibits a recall-dominated profile: it achieves near-perfect recall (0.97–1.0 across all drift types and abruptness conditions), meaning it detects almost every drift. However, its precision is among the lowest (0.05–0.08), indicating that the majority of its alarms are false positives. This pattern suggests that this method is overly sensitive, triggering alarms frequently regardless of whether a genuine drift has occurred. In practice, however, this kind of behavior leads to excessive and unnecessary model retraining, undermining trust in the detection system. Other detectors such as RDDM exhibit a similar profile, achieving high recall but low precision. SEED, ABCD, and STEPD show a relatively balanced precision and recall trade-off across all scenarios. Finally, under gradual drift conditions, overall scores are lower than for abrupt drifts, but the relative performance of detectors is largely preserved.

Figure 6 displays the average execution time (in minutes) for each drift detection method, aggregated across all datasets. The execution time encompasses the complete detection pipeline. This metric complements the main results by providing insight into the computational overhead associated with each detector. The computational cost varies across methods. Most methods show a similar cost, except for ABCD, ABCD(X), and STUDD which are more expensive.

Table 4: Detector Parameter Search Space

Detector	Parameter	Values
ABCD	delta_drift	0.001, 0.002, 0.0001, 0.01, 0.02
	delta_warn	0.1, 0.01
	encoding_factor	0.3, 0.5, 0.7
	model_id	kpca, ae
	num_splits	20, 50, 100
ABCD(X)	delta_drift	0.001, 0.002, 0.0001, 0.01, 0.02
	delta_warn	0.1, 0.01
	encoding_factor	0.3, 0.5, 0.7
	model_id	kpca, ae
	num_splits	20, 50, 100
ADWIN	delta	0.001, 0.002, 0.005, 0.01, 0.0005, 0.0001
CUSUM	min_n_instances	30, 50, 100, 300, 500, 1000, 2000
	delta	0.001, 0.002, 0.005, 0.0001, 0.01
	lambda_	50, 100, 150, 300, 500, 1000, 2000
DDM	min_n_instances	30, 50, 100, 300, 500, 1000, 2000
	out_control_level	1.75, 2, 2.5, 3.0, 2.25, 2.75, 3.5, 4
EWMA	min_n_instances	50, 100, 300, 500, 1000, 2000, 3000, 5000, 10000
	lambda_	0.9, 0.01, 0.001, 0.1, 0.005, 0.002, 0.0001
GMA	min_n_instances	30, 50, 100, 300, 500, 1000, 2000
	lambda_	0.001, 0.002, 0.01, 0.1, 0.5, 1, 2, 3, 5
	alpha	0.99, 0.995, 0.9, 0.8, 0.7, 0.5, 0.1, 0.01
HDDMA	drift_confidence	0.001, 0.002, 0.005, 0.01, 0.0001
	test_type	Two-sided, One-sided
HDDMW	drift_confidence	0.001, 0.002, 0.005, 0.01, 0.0001
	test_type	Two-sided, One-sided
	lambda_	0.05, 0.001, 0.1, 0.01, 0.0001
PH	min_n_instances	30, 50, 100, 300, 500, 1000, 2000
	delta	0.001, 0.002, 0.005, 0.01, 0.0001, 0.1
	lambda_	30, 50, 100, 300, 500, 1000, 2000
	alpha	0.99, 0.999, 0.995, 0.9, 0.8, 0.5
RDDM	min_n_instances	30, 50, 100, 300, 500, 1000, 2000
	drift_level	1.9, 2, 2.1, 2.25, 2.5, 3, 1.5, 1.75
SEED	delta	0.0001, 0.001, 0.01, 0.05, 0.1
	epsilon_prime	0.0025, 0.01, 0.005, 0.0075
	block_size	32, 50, 100, 150, 256
	alpha	0.2, 0.3, 0.4, 0.5, 0.6, 0.7, 0.8
STEPD	window_size	30, 50, 100, 300, 500, 1000
	alpha_drift	0.001, 0.002, 0.003, 0.005, 0.01, 0.1
STUDD	min_n_instances	500, 1000, 2000, 5000

Table 11 reports F1 scores on three synthetic data stream generators (Agrawal, SEA, STAGGER), each composed of either abrupt or gradual drifts, using only the error-tracking detectors. The overall ranking of detectors on synthetic streams broadly corroborates the findings on real-world data: SEED, ABCD, and STEPD achieve the highest F1 scores across most synthetic scenarios.

Table 12 reports F1 scores for five classifiers—Adaptive Random Forest (ARF), Hoeffding Tree, Naive Bayes, Online Bagging, and OzaBoost—on the Electricity dataset using the label swap drift type. This experiment assesses whether the relative performance of detectors is sensitive to the underlying learning algorithm. The relative ranking of detectors is largely preserved across classifiers. This stability suggests that the findings from our main experiments, which use the Hoeffding Tree, are not a result of a particular learner's error dynamics. Absolute F1 scores, however, vary noticeably across classifiers. OzaBoost consistently yields the highest detection scores (e.g., 0.82 for SEED, 0.78 for STEPD and HDDMA), while others tend to produce lower scores.

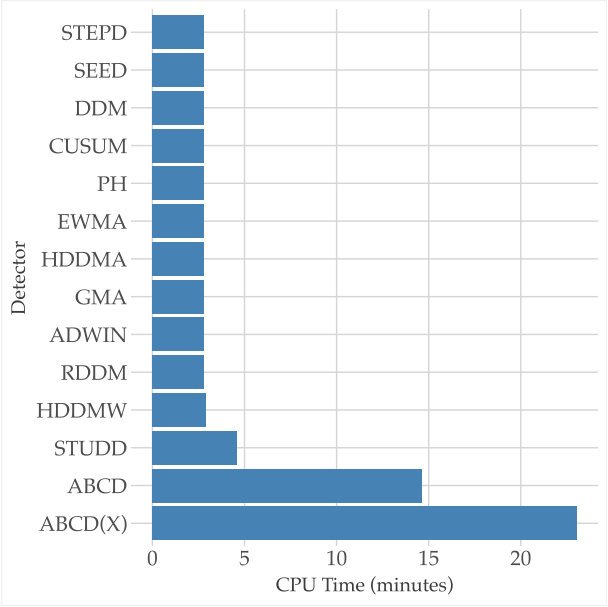


Figure 6: Average execution time of each drift detection method across all datasets

Table 5: Average F1 detection score of drift detectors across different datasets for abrupt drifts

Detector	Feature Filtering	Feat. Permutation	Class Prior	Class Swap
ABCD	0.11	0.51	0.16	0.25
ABCD(X)	0.02	**0.98**	0.0	0.0
ADWIN	0.24	0.27	**0.31**	0.16
CUSUM	0.19	0.01	0.22	0.1
DDM	0.16	0.02	0.19	0.09
EWMA	0.09	0.0	0.06	0.03
GMA	0.1	0.13	0.12	0.15
HDDMA	0.17	0.36	0.2	0.24
HDDMW	0.18	0.25	0.19	0.21
PH	0.02	0.0	0.06	0.09
RDDM	0.2	0.04	0.15	0.1
SEED	0.24	0.61	0.3	**0.37**
STEPD	**0.26**	0.53	0.28	0.33
STUDD	0.17	0.27	0.1	0.11

Table 8: Average precision score of drift detectors across different datasets for gradual drifts

Detector	Feature Filtering	Feat. Permutation	Class Prior	Class Swap
ABCD	0.03	0.26	0.08	0.17
ABCD(X)	0.0	**0.66**	0.0	0.0
ADWIN	0.07	0.03	0.1	0.05
CUSUM	0.1	0.02	0.13	0.08
DDM	0.07	0.04	0.08	0.06
EWMA	0.05	0.0	0.05	0.04
GMA	0.07	0.07	0.07	0.08
HDDMA	0.06	0.09	0.13	0.05
HDDMW	0.07	0.08	0.07	0.09
PH	0.06	0.01	0.12	0.05
RDDM	0.07	0.07	0.07	0.08
SEED	0.09	0.33	0.13	0.23
STEPD	**0.11**	0.33	0.16	**0.23**
STUDD	0.09	0.1	0.09	0.06

Table 9: Average recall score of drift detectors across different datasets for abrupt drifts

Detector	Feature Filtering	Feat. Permutation	Class Prior	Class Swap
ABCD	0.14	0.7	0.17	0.37
ABCD(X)	0.01	0.96	0.0	0.0
ADWIN	0.4	0.55	0.51	0.47
CUSUM	0.23	0.02	0.26	0.18
DDM	0.23	0.04	0.29	0.21
EWMA	0.1	0.0	0.08	0.05
GMA	**0.97**	**1.0**	**1.0**	1.0
HDDMA	0.23	0.66	0.3	0.45
HDDMW	0.33	0.67	0.45	0.54
PH	0.02	0.0	0.04	0.07
RDDM	0.33	0.11	0.32	0.29
SEED	0.29	0.93	0.36	0.58
STEPD	0.36	0.92	0.39	0.58
STUDD	0.48	0.75	0.38	0.41

Table 10: Average recall score of drift detectors across different datasets for gradual drifts

Detector	Feature Filtering	Feat. Permutation	Class Prior	Class Swap
ABCD	0.04	0.5	0.08	0.28
ABCD(X)	0.0	0.76	0.0	0.0
ADWIN	0.24	0.1	0.34	0.27
CUSUM	0.14	0.02	0.17	0.11
DDM	0.89	0.69	0.89	0.78
EWMA	0.06	0.01	0.06	0.03
GMA	**1.0**	**1.0**	**1.0**	**1.0**
HDDMA	0.09	0.17	0.15	0.11
HDDMW	0.65	0.78	0.66	0.81
PH	0.05	0.01	0.09	0.05
RDDM	**1.0**	0.99	1.0	**1.0**
SEED	0.19	0.87	0.27	0.59
STEPD	0.19	0.84	0.28	0.54
STUDD	0.43	0.61	0.37	0.41

Table 11: F1 scores on synthetic data streams (Agrawal, SEA, STAGGER) composed of abrupt or gradual drifts.

Mode	ABRUPT			GRADUAL		
Stream	Agrawal	SEA	STAGGER	Agrawal	SEA	STAGGER
ABCD	0.57	0.07	**0.40**	0.60	0.33	0.44
ADWIN	0.26	0.04	0.04	0.54	0.54	0.08
CUSUM	0.12	0.00	0.04	0.17	0.22	0.03
DDM	0.10	0.00	0.04	0.34	0.36	0.11
EWMA	0.00	0.00	0.00	0.00	0.00	0.00
GMA	0.18	0.19	0.16	0.40	0.39	0.45
HDDMA	0.22	0.05	0.12	0.32	0.26	0.16
HDDMW	0.16	0.09	0.05	0.45	0.39	**0.50**
PH	0.06	0.00	0.00	0.14	0.07	0.03
RDDM	0.15	0.08	0.04	0.40	0.43	0.41
SEED	**0.58**	0.46	0.35	**0.61**	**0.63**	0.47
STEPD	0.49	**0.49**	0.29	**0.61**	0.61	0.42

Table 12: F1 scores of each detector for different classifiers using the Electricity data stream and the label swapping drift type.

Learner	ARF	HoeffdingTree	NaiveBayes	OnlineBagging	OzaBoost
ABCD	0.51	0.33	**0.53**	0.35	0.52
ADWIN	0.23	0.19	0.44	0.22	0.68
CUSUM	0.0	0.1	0.37	0.01	0.6
DDM	0.03	0.06	0.25	0.06	0.4
EWMA	0.0	0.0	0.0	0.0	0.0
GMA	0.2	0.18	0.15	0.2	0.17
HDDMA	0.56	0.51	0.41	0.47	0.78
HDDMW	**0.62**	0.42	0.22	0.46	0.63
PH	0.0	0.02	0.06	0.0	0.78
RDDM	0.06	0.12	0.18	0.08	0.3
SEED	0.51	**0.54**	0.5	0.53	**0.82**
STEPD	0.57	**0.54**	0.46	**0.56**	0.78